

Cross-check of airframe_final_v2

What was checked, how, and what was found. Every claim below was re-run from the captures or the code; nothing was taken from the v2 documents on trust.

1. What changed in v2

Compared file by file with the previous bundle:

- **Rewritten:** the fast pipeline (`airframe_analyze.py` grew from 56 KB to 103 KB), plus its dashboard, alerts and `CHANGES.md`.
- **New:** `pipeline_fast/tests/check_fast.py` (48 checks), `results/fast_pipeline_findings.csv` and `results/fast_pipeline_tests.txt`.
- **Updated:** the README, the project summary, the dashboard and both PDFs, mainly the speed and scale numbers.
- **Unchanged:** the Wireshark-based pipeline and its tests, and the original uploads.

2. v2's claims, re-run

Claim in v2	Result
All existing numbers unchanged: 703 attempts, same outcomes, 1,317 + 141 reason-2 frames, 63 clients, 26 APs	✓ Identical outcome counts to the original upload
48/48 checks pass	✓ Re-run: 48/48
Shipped results match the engine	✓ <code>findings.csv</code> identical to a fresh run; the report differs only in the order of tied clients, which depends on the salt
Every file in <code>results/</code> is masked	✓ None of the 152 real MAC addresses or 2 real SSIDs appears in any v2 file
MACs keep the vendor prefix plus a 40-bit keyed hash; EAP identities never read	✓ HMAC-SHA256, 10 hex characters; the parser never reads identity text
About 4.5 s on one core, about 250,000 frames/s	✓ 4.4 s on a warm run (254,000 frames/s). A cold first run took 6.9 s
2.5 s on two cores	Not verified: the test machine has one core
Edge files "about 2,000x smaller" with identical results	Partly: identical results ✓, but the measured ratio is 1,807x (0.16 MB from 286 MB). Corrected in <code>CHANGES.md</code>
No raw identifiers in edge files with <code>--mask</code>	✓ Checked both as text and as raw bytes

Claim in v2	Result
Same result on 1 or 32 partitions	✓ Identical attempts, findings and disconnects
96 sensors give the same result on 1 or 32 partitions	Not verified

3. Data cross-check

Headline numbers were recomputed directly from the raw frames, independently of both engines, and then compared with each engine's output. All match.

Number	Raw frames	Fast (v2)	Wireshark-based
Frames, sensors, channels	1,118,853 / 8 / 8	same	same
802.1X clients failing, APs	63 / 26	63 / 26	63 / 26
EAP codes seen	Request and Response only; all requests are Identity	same	same
Successful joins	23 IoT devices with M3/M4	5 + 26 probable = 31 attempts	31 attempts
Rejection wave	18 Intel clients, 18 APs, from 14:30:53	18, 14:30:53	18
Reason-2 frames (after merging copies)	1,477 raw; 1,458 after removing retransmissions	1,317 + 141	1,458
IoT handshake stuck at M1	1, at 14:27:21	1	1

4. New sample cases: 11 cases, 24 checks, both engines

`sample_cases/sample_cases.py` builds eleven multi-sensor captures that neither existing test suite contains. `run_sample_cases.py` runs both engines on each and checks them against the known answer.

Case	What it tests
<code>partial_credential_reject</code>	3 of 10 clients refused: per-client findings, but not "network-wide"
<code>sensor_gap</code>	A sensor stops recording for 2 minutes: that must not look like silent APs
<code>clock_skew_same_channel</code>	Two sensors hear the same failures with a 0.3 s clock offset: one finding, 3 attempts
<code>spoofed_flood</code>	200 deauths with random sequence numbers: called a flood, never "sent by the AP"
<code>ping_pong_roam</code>	9 successful roams between two channels: no findings, 9 roams counted

Case	What it tests
hidden_ssid_wrong_psk	Hidden network: a wrong-passphrase client is flagged, the other client is not
randomized_mac_timeout	A randomised MAC is still tracked, and masked output has no raw MAC or SSID
mixed_incident	802.1X outage on 4 APs + loop on the PSK network + congestion on ch 44, kept apart
eap_ok_handshake_timeout	Login succeeds, then the key handshake times out: a handshake finding, not 802.1X
already_connected_then_leaves	Client joined before the capture started and then leaves: nothing to report
retry_mix_trap	Probe responses retried 31%, data 1%: no congestion

Results:

- Fast pipeline (v2): **24/24**.
- Wireshark-based pipeline: **19/24 before fixes, 24/24 after** (section 6).

5. Issues found in v2

1. **Masking is off by default.** Without `--mask`, the fast pipeline writes raw MACs to the report, CSVs and dashboard, and nothing in the report says so. The earlier patch masked by default and printed a warning when it didn't. Recommendation: make masking the default and keep an explicit `--no-mask` for debugging. Until then, always run the demo with `--mask`.
2. **Two earlier honesty corrections were lost** in the dashboard and PDFs. The opening line no longer says "Replaying the captures", so it reads as if the 5-second alert was live. It is restored in this version. The ZK survey is again described as "100 administrators"; sources report 100 or 102, a minor point.
3. **Edge ratio overstated:** "about 2,000x" is 1,807x as measured. Corrected in `CHANGES.md`.
4. **Two "cannot join" info findings on the challenge data** are attempts cut short by the rejection wave (reason 2 before any login started), not separate join problems. Minor, since both are info level.
5. **Differences between the engines to know about (not errors):**
 - The fast pipeline has no co-channel-overlap or join-burst finding. The "channels 36 and 40 carry 5 APs" and "24 IoT rejoins" findings come from the Wireshark-based pipeline.
 - Probe storms are defined differently. Fast counts 10+ probe requests on one channel in 10 s and finds none in the challenge data. The Wireshark-based pipeline counts 15+ across all channels and finds 8.
 - The stuck IoT handshake is "info" in fast and "high" in the Wireshark-based pipeline.

6. Fixes made in this version (Wireshark-based pipeline)

The sample cases exposed four real bugs in `pipeline_tshark/detect.py`. All are fixed.

1. Different APs merged into one.

- The bug: APs whose MAC addresses differed only in the last byte were treated as one radio. In `mixed_incident` that hid a network-wide outage on 4 APs as "1 AP".
- The fix: BSSIDs are now grouped only if they share a MAC block and a channel and advertise different SSIDs.

2. Clock skew doubled events.

- The bug: the same frame heard by two sensors 0.3 s apart counted twice.
- The fix: the merge window is 1 s instead of 50 ms. Same addresses, type and sequence number within 1 s is the same frame.

3. Roams missed.

- The bug: a reassociation to another AP counted as a roam only within 10 s of the last frame seen.
- The fix: a reassociation is now always recognised as a roam.

4. Sensor gaps reported as silent APs and congestion.

- The bug: a sensor that stopped recording made every AP look silent.
- The fix: gaps longer than 5 s are detected and reported as a "Sensor gap" info finding. Beacon verdicts use only time the sensor was recording.

Regression check:

- All 56 tests pass.
- On the challenge data, the 95 findings and all outcomes are identical to before.
- 29 APs, as before.

7. Not verified

- The two-core timing and the 96-sensor partition test: the test machine has one core.
- Behaviour on live traffic: everything here is replayed captures.
- Business impact, which the captures cannot show.

8. Follow-up in v4 (fast pipeline)

The v4 bundle answers the issues in section 5. Details are in `pipeline_fast/CHANGES.md`.

- **Issue 1, masking:** now on by default. `--no-mask` is explicit, and the output then says "do not share".
- **Issue 4, the two "cannot join" findings:** they are now counted inside the rejection-wave finding.
- **Issue 5a, missing finding types:** co-channel overlap and join burst are added with the same rules as the Wireshark-based pipeline. On the challenge data both engines now agree: channels 36 and 40 (5 APs each), and the 24-client IoT-network burst.
- **Issue 5b, probe-storm definitions:** they still differ, on purpose. The fast engine flags a client hammering one channel. Clients rescanning after being kicked show up as the fleet-level probe

storm.

- **Issue 5c, stuck-handshake severity:** unchanged. One attempt at M1 is info in the fast engine.
- **Section 7, items not verified:** both are now verified on a 2-core machine.
 - Two cores: 2.5 s.
 - 96 sensors: identical attempts and findings on 1 or 32 partitions, matching the 8-sensor run.

The v4 checks:

- 48/48 in `check_fast.py`;
- 24/24 for the fast engine in the sample cases;
- the challenge-data numbers are unchanged.

The Wireshark-based pipeline was not re-run in v4 (no tshark on the machine used), and its code is unchanged from v3.